

LAIRS

Unified Technical Standard

Local Autonomous Intelligence and Reasoning System

The open standard for Runtime-Governed Systems — with LAIRS as the reference implementation.

Russell Aydelotte

Aegis Secure Convergence Systems

v2.0 · July 24, 2026 · Supersedes v1.0

Public — Not Confidential — Not Proprietary

"Intelligence is local. Authority remains human. Evolution is deliberate."

© 2026 Russell Aydelotte, Aegis Secure Convergence Systems. Licensed under CC BY-NC-SA 4.0.

This document is the open, non-proprietary convergence of the **LAIRS Runtime Governance Architecture** and the **LAIRS Audit Fabric Standard** into a single public-release technical reference. It defines the **Runtime-Governed Systems** standard — a category of computing system — and specifies LAIRS as the reference implementation that satisfies it. All content is approved for unrestricted distribution under the license stated above.

Intelligence is local. Authority remains human. Evolution is deliberate.

Platform Status — Active Research Deployment

- LAIRS is an active research and architectural platform. It is more mature than the v1.0 two-node prototype: a hardware root of trust, post-quantum signing, and governed change-control are now operational (see §23). It is **not yet production-commissioned** and has **not undergone external adversarial review** — a prerequisite for any customer deployment.
- Intended for controlled experimentation. Components marked *In Development* are under active work.
- No component acts autonomously or irreversibly without explicit Human Master Administrator cryptographic approval. Do not deploy in production without that authorization.

What Changed Since v1.0

v1.0 described a two-node observe-only prototype and listed a hardware root of trust and post-quantum cryptography as *roadmap* items. Both are now real. This revision reflects: the **ANVIL** hardware-root-of-trust signing appliance; a **post-quantum cryptographic foundation (SPECTER)** with ML-DSA-87 / ML-KEM-1024 (post-quantum signing live; hardware-rooted hybrid AND-signing at the ANVIL trust root, staged in observe — see §6, §23); **Sentinel Forge** moved from *In Development* to *deployed*; the **Execution Provenance Gate (EPG)**; the sovereign reasoning model **PRAETOR** and its continuity substrate **Mnemos**; a **three-node topology** (DL360, Lenovo, ANVIL) with SentiNet policy bundle v2.2; a **two-authority model** (ANVIL = signing authority; LICTOR = enforcement authority, *in development*); and a **staged enforcement posture** (enforcing where proven, observing elsewhere).

Nomenclature — Component Names

LAIRS component names are **acronyms**; each is expanded here and on first use in the text.

Acronym	Expansion
LAIRS	Local Autonomous Intelligence and Reasoning System
ANVIL	Attested Node, Vaulted Identity Lineage
SPECTER	Stateless Protocol for Ephemeral Cryptographic Token Endorsement and Revocation
LICTOR	Latching Interlock & Containment, Trusted Operational Response
GAT	Governed Action Token
GAE	Governed Adversarial Evolution
EPG	Execution Provenance Gate

A second class of components carry **descriptive names** — the name itself states the module's purpose rather than encoding an acronym: **PRAETOR** (a Roman magistrate holding *imperium*, delegated command authority → the sovereign reasoning seat), **VEDETTE** (a mounted sentry posted forward of an outpost → the forward OSINT scout), **Mnemos** (from Mnemosyne, memory personified → the persistent-continuity spine), **Metatron's Gate** (the recording scribe → the authority and decision ledger), and **SentiNet** (Sentinel + Network → the verification fabric).

Table of Contents

Part	Sections	Title
I	§1–§14	Runtime Governance Architecture
II	§15–§26	Audit Fabric Standard
III	§27	Component Catalog
IV	§28–§36	Deployment & Technical Reference
—	Appendix A	Terminology
—	Appendix B	References
—	Appendix C	Related Standards and Positioning

PART I — Runtime Governance Architecture

Defines the LAIRS standard for **Runtime-Governed Systems**: computing environments in which authority, behavior, and state changes are continuously validated through cryptographic proof, independent verification, and enforceable policy constraints *during operation*. This part covers the governance model, the required layers, the cryptographic foundation, the hardware trust anchor, artifact specifications, and compliance criteria.

§1 — Definition

A **Runtime-Governed System** is a computing environment in which authority, behavior, and state changes are continuously validated through cryptographic proof, independent verification, and enforceable policy constraints during operation — not only at deployment time. Unlike systems that rely on static configuration trust, a runtime-governed system maintains persistent assurance of correctness, authorization, and integrity while executing.

Core Principle — Trust is not assumed. Trust is continuously proven.

§2 — Problems Addressed

Runtime governance directly addresses structural weaknesses of modern distributed and autonomous systems by introducing verifiable authority loops:

- Configuration and policy drift over time
- Loss of authority provenance for system actions
- Inability to independently verify state legitimacy
- Weak coupling between decision and enforcement layers
- Over-reliance on perimeter or identity-only security models
- Lack of provable governance for automated or AI-driven changes
- **Software-only trust roots** that cannot attest to the state that produced a signature

§3 — Governance Model — Layered Architecture

The LAIRS architecture defines **three governance layers** anchored beneath by a **hardware root of trust**. Each layer answers a distinct trust question at runtime.

Layer	Name	Responsibilities	Trust Question
0 · Trust Root	ANVIL (§13)	Hardware-attested signing authority; TPM-sealed keys; measured boot; issues the cryptographic root every higher layer verifies against	"Is the authority itself genuine and unmodified?"
1 · Authority	Metatron's Gate	Issue signed GATs; immutable decision ledger; quorum/approval enforcement; anchor trust roots; audit provenance	"Who authorized this action?"

Layer	Name	Responsibilities	Trust Question
2 · Fabric	SentiNet	Independent artifact verification; node-binding enforcement; signature validation; Last-Known-Good state; cross-node trust propagation	"Can this authority be trusted here?"
3 · Enforcement	Sentinel Agents + EPG	Runtime policy enforcement; execution-provenance gating; integrity monitoring; behavior control; evidence generation; local protective response	"Is the system doing only what was authorized?"

Two-Authority Separation. Signing authority (ANVIL) and enforcement authority (**LICTOR**, §14 — *In Development*) are deliberately separate roles on separate trust domains. The entity that *authorizes* an action is never the entity that *executes* it, and neither holds the other's keys.

§4 — Governed Action Token (GAT)

A runtime-governed system authorizes all actions through an explicit, cryptographically signed **Governed Action Token (GAT)** — the universal authorization primitive for any governed change, action, or enforcement. Without a valid, verified GAT, enforcement must **fail closed** — no exceptions.

Property	Purpose
Cryptographically signed	Tamper-evident authenticity from the issuing authority
Hardware-rooted, hybrid signature	<i>Target scheme</i> — a classical AND post-quantum pair (ECDSA-P256 + ML-DSA-87) rooted in the ANVIL TPM; produced by ANVIL and staged in observe today (§4.1, §23)
Time bounded	Tokens expire — stale authority cannot be replayed
Context scoped	Authority limited to specific action types and contexts
Node bound	Tokens bind to specific infrastructure nodes or identities
Independently verifiable	Third-party validation without querying the issuer, offline, against pinned keys
Ledger anchored	Linked to an immutable decision log for provenance
Authority metadata	Records issuer, governing policy, and approval quorum
Action intent hash	Binds the token to the exact action being authorized

§4.1 — Hardware-Rooted GAT Signing

Each GAT is signed over `sha256(canonical(token))` by the ANVIL signing authority, producing a **hybrid AND-signature**: an ECDSA-P256 half and an ML-DSA-87 half, both of which must verify. The signature is **attestation-bound** — ANVIL returns a TPM quote proving the genuine appliance in its expected measured state produced it. Verifiers check both halves offline against locally pinned public keys, so verification has no runtime dependency on the signer. (*Migration status: hardware-rooted hybrid signing is deployed and verified in observe mode alongside the legacy RSA-PSS signature; the cutover to hardware-root-required is staged — see §23, §36.*)

§5 — Runtime Verification Requirements

A system qualifies as runtime-governed only if all of the following hold continuously during operation:

- Authority artifacts are validated during execution — not only at deployment
- Verification occurs **independently** from the issuing authority
- Enforcement modules refuse actions when validation fails
- Expired or revoked authority cannot execute under any condition
- State rollback / Last-Known-Good (LKG) protection exists and is tested
- Evidence of every decision is recorded immutably
- Nodes can prove their operational state to external observers on demand
- **Signing authority can attest to the hardware state that produced each signature**

§6 — Post-Quantum Cryptographic Foundation (SPECTER)

LAIRS signing is **post-quantum-aligned by policy** — every enforcement token uses post-quantum cryptography (PQC); no trust-based shortcuts, no classical-only authority. The cryptographic layer is **SPECTER**, the first formally-specified instantiation of the *Ethereal Cryptography* paradigm.

Primitive	Role
ML-DSA-87 (FIPS 204)	Post-quantum signatures on GATs, audit segments, policy bundles, model lineage
ML-KEM-1024 (FIPS 203)	Post-quantum key encapsulation — the lattice-seed layer of the Ghost Key
Hybrid AND-signing	Realized at the ANVIL hardware root : ECDSA-P256 and ML-DSA-87, both of which must verify (§4.1). Deployed in <i>observe</i> mode today; the hybrid-required cutover is staged (§36)
SPECTER Ghost Key ceremony	A single-use signing key assembled from Shamir shares and a hardware-KEM lattice seed (composed via HKDF-SHA3-512), then destroyed; the key never persists at rest

Where each layer composes today (verified against the live deployment, July 2026). Post-quantum *signing* is live: under `require_pqc`, the SPECTER Ghost Key emits a single **ML-DSA-87** signature and fails rather than degrade to a classical one — it does **not** itself emit a composed classical+PQC signature. The genuine hybrid

AND-signature (both halves required) is produced by the **ANVIL hardware root** and currently runs in *observe* mode alongside the legacy RSA-PSS base signature. The Ghost Key's **KEM-seed** layer is genuinely hybrid (Shamir secret **and** ML-KEM-1024, combined via HKDF-SHA3-512). The single unified hybrid-AND *signature* becomes the required scheme at the ANVIL custody cutover (§36).

§7 — Runtime Governance Loop

Every governed action traverses the same loop:

Intent → Metatron's Gate (classify · quorum · sign GAT via ANVIL) → SentiNet (verify authority is valid here) → Enforcement (Sentinel + EPG: act only under a live, verified GAT) → Audit Fabric (seal the decision provenance) → SentiNet (propagate signed state)

No stage trusts the previous stage's word; each re-verifies cryptographically.

§8 — Governed Adversarial Evolution (GAE)

LAIRS enables continuous improvement without sacrificing governance through **GAE** (*deployed*, red/blue/purple exercise engine). GAE defines the policy envelope within which adversarial and defensive activity may occur, with human approval gates on all system changes.

Component	Description
Red Team Pressure	Adversarial testing inside defined policy envelopes
Blue Team Defense	Adaptive defense within governed boundaries
Purple Team Synthesis	Coordinated offensive/defensive findings
AI Advisory	AI recommendations without autonomous authority to act
Human Approval Gates	Cryptographic human sign-off required for all changes
Replayable Scenarios	Scenarios recorded for regression testing and audit

Deterministic proposals (detection rules, policy, config) flow to Sentinel Forge for governed apply; code changes are human-authored from GAE findings. Repository: reference [5], Appendix B.

§9 — Staged Enforcement (Observe → Enforce)

Runtime-governed systems support **staged deployment**: each control is proven in observe mode before it enforces. LAIRS runs a mixed posture — not a single global mode.

- **Enforcing today:** Metatron's Gate (ENFORCING; `require_pqc` on); Sentinel Forge governed apply (signature-re-verifying, fail-closed); Praetorian Guard; the signed Sentinel organic baseline.
- **Observe / staged:** hardware-root-required GAT verification (observing alongside RSA); SentiNet network *enforcement* (policy is distributed and verified; egress/zone enforcement is Phase 3); LICTOR enforcement authority (not yet built).

In every observe stage the verification pipeline is fully active and generating evidence; no destructive action is permitted on a failed verification.

§10 — Properties of a Runtime-Governed System

A compliant system demonstrates all of the following, each paired with its safety complement:

Property	Complementary Property
Verifiable authority provenance	Fail-closed behavior under uncertainty
Continuous policy validation	Bounded autonomy
Cryptographic trust anchors	Human override capability
Hardware-attested signing root	Recoverable safe state (LKG)
Independent enforcement capability	Deterministic auditability
Post-quantum-aligned signatures	Cross-node trust coherence

§11 — Comparison With Traditional Security Models

Dimension	Traditional Security	LAIRS Runtime Governance
Trust model	Static configuration trust	Continuous cryptographic verification
Authorization	Identity-based	Artifact-based authority (GAT)
Trust root	Software / key-on-disk	Hardware-attested (ANVIL TPM)
Cryptography	Classical (RSA/ECDSA)	Post-quantum (ML-DSA-87) live; hardware hybrid-AND staged at the trust root
Control topology	Centralized control plane	Distributed verification fabric
Audit timing	Logs after the fact	Proof generated during execution
Failure behavior	Undefined / fail-open risk	Fail-closed by design
Adaptation	Reactive response	Governed adversarial evolution

§12 — Compliance Criteria (The Standard)

A system meets the **LAIRS Runtime Governance Standard** if it satisfies all of the following. This is the conformance rubric for the category; LAIRS is its reference implementation.

1. Uses cryptographically signed governance artifacts (GATs) for all authorized actions
2. Signs authority with a **post-quantum-aligned** scheme (hybrid classical-AND-PQC acceptable; classical-only is non-conformant)
3. Roots signing authority in a **hardware trust anchor** that can attest to its own measured state
4. Performs **independent** runtime verification across all enforcement nodes
5. Enforces policy through verifiable agents with **fail-closed** behavior on artifact failure
6. Maintains immutable decision provenance via a cryptographically anchored ledger
7. Supports safe recovery and Last-Known-Good state rollback
8. Enables governed evolution without authority loss (GAE)
9. Implements **multi-authority quorum** governance for high-assurance decisions
10. **Separates signing authority from enforcement authority** across distinct trust domains

Reference-implementation self-assessment (stated, not hidden). LAIRS meets criteria 1, 3, 4, and 6–9 in the current deployment. **Three criteria are only partially met:**

- **Criterion 2** (post-quantum-aligned signing) is met on the **GAT / audit-segment** path — ML-DSA-87 is live (§6, §23) — but **not yet on the SentiNet policy-bundle path**, which is **Ed25519-only** today (§21). The hybrid Ed25519 × ML-DSA-87 migration (§36) closes it.
- **Criterion 5** (fail-closed enforcement by verifiable agents) is met by the **enforcing** components — Metatron's Gate, Sentinel Forge, and the Praetorian Guard, all fail-closed — while **host-level Sentinel enforcement runs in observe/staged mode** (§9, §23). No destructive action is permitted on a failed verification, but full agent-level enforcement is staged.
- **Criterion 10** (separation of signing and enforcement authority) holds *in practice* — signing keys are off-host on ANVIL and enforcement is performed by separate components (Sentinel Agents + EPG) — but the dedicated **two-authority model is incomplete: LICTOR**, the enforcement authority, is specified and **not yet built** (§14, §36).

A conformance claim for LAIRS must carry all three caveats.

§13 — Hardware Root of Trust (ANVIL)

ANVIL is the signing-authority appliance — the trust anchor beneath the governance layers. *(No longer roadmap; deployed as of this revision, production appliance in progress.)*

- **Off-host signing authority.** A dedicated appliance (TPM-equipped) reachable only over an isolated link, so the signing key never resides on any enforcement host. It exposes attestation, signing, and key-encapsulation over mutual TLS.
- **Attestation-first.** Every signature is accompanied by a TPM quote binding it to the genuine appliance in its expected measured state; verifiers pin the appliance identity, not a TLS hostname.
- **Sealed keys.** Signing keys (ECDSA-P256 + ML-DSA-87) and the KEM key are TPM-sealed to boot-state PCRs — unsealable only on the genuine appliance in its expected state.
- **Production posture (in progress):** a custom immutable appliance OS — secure boot (key hash in silicon OTP) → measured boot → dm-verity read-only rootfs → TPM-sealed keys → LUKS data → A/B atomic updates → reproducible build for provenance and adversarial review.

§14 — Enforcement Authority (LICTOR) (In Development)

LICTOR is the counterpart to ANVIL: the latching interlock and containment authority that *executes* enforcement decisions. Actions are routed to LICTOR by class; high-consequence actions require ANVIL-signed authorization that LICTOR verifies before acting. Separation of the signing and enforcing roles is a compliance requirement (§12.10). *Named and specified; not yet built.*

PART II — Audit Fabric Standard

Deterministic, segmented, cryptographically anchored logging for governed systems with distributed verification. The **LAIRS Audit Fabric** combines governance-aware events, segmented Merkle sealing, and bounded-lifecycle storage, extended through SentiNet for distributed verification.

§15 — Problem Statement

Existing audit mechanisms cannot simultaneously provide: tamper-evident continuity under adversarial conditions; governance-aware context native to each record; bounded storage lifecycle with preserved integrity; offline/independent verification; distributed trust propagation; and fail-closed guarantees under partial failure. Blockchain addresses decentralized trust but adds consensus complexity, unbounded storage growth, and a semantic mismatch unsuitable for infrastructure telemetry and runtime governance. The LAIRS Audit Fabric resolves these conflicts.

§16 — Architectural Overview

Five integrated planes. Each is defined below; their runtime sequence is given by the event pipeline in §17, and their trust-layer relationship by the layer hierarchy in §3 — the table below is a catalog of the planes, not an ordering:

Plane	Component	Role
Trust Root	ANVIL	Hardware-attested signing authority for artifacts
Enforcement	Sentinel Agent + EPG	Runtime policy enforcement, execution-provenance gating, event generation
Governance	Metatron's Gate	Authority validation, GAT issuance, quorum decisions
Audit Fabric	Ledger + Merkle	Cryptographic event anchoring and segment sealing
Verification	SentiNet	Distributed trust propagation and node integrity

§17 — Deterministic Event Pipeline

Events flow through an ordered pipeline from runtime origin to verified storage; each stage produces verifiable output consumed by the next:

1. **Runtime event** — Sentinel Agent generates an enforcement event
2. **Sentinel enforcement** — policy evaluation (pass / fail / record); **EPG** gates execution against verified provenance
3. **Governance validation** — Metatron's Gate: GAT check, quorum verification
4. **Ledger event construction** — embed governance context, actor identity, intent

5. **Hash-chain continuity** — link to previous event hash (tamper-evident chain)
6. **Segmented Merkle sealing** — compute the Merkle tree over the segment, generate the root
7. **Signed artifact storage** — apply the ANVIL-rooted PQC signature, commit the artifact
8. **SentiNet distribution** — push the signed artifact to verification nodes
9. **Node verification** — independent signature and root validation
10. **Tiered storage lifecycle** — Hot → Warm → Cold, integrity preserved

§18 — Segmented Merkle Ledger Model

The fabric uses **segmented** Merkle ledgers rather than a monolithic append-only log. Events are grouped into discrete segments that undergo deterministic sealing, producing portable, independently verifiable trust artifacts and enabling partial verification of history without the full log:

- Merkle tree over all events in a segment
- Root generation — a canonical segment fingerprint (SHA-256)
- Payload hashing binding compressed content to the seal
- **PQC digital signing** by the governance authority, rooted in ANVIL
- Ledger commitment — an immutable provenance anchor with `prev_hash` chain

§19 — Governance-Coupled Event Semantics

Audit events are not passive log entries — each carries embedded governance context, making it a **decision provenance record**. This distinguishes the fabric from conventional SIEM/syslog.

Field	Purpose
Actor Identity	Cryptographically verified identity of the acting principal
Role Classification	Authority classification of the actor at time of action
Intent Metadata	Declared purpose of the action for governance traceability
Policy Constraints	Active policy envelope governing the action
Approval State	Quorum or unilateral approval status with signatory list
System Context	Runtime state snapshot at event creation

§20 — Execution Provenance Gate (EPG)

EPG binds *what runs* to *what was authorized*. Before a governed execution proceeds, EPG checks it against a signed provenance record and refuses execution whose provenance cannot be verified — extending fail-closed governance from *authority* (the GAT) to *execution* itself. EPG maintains its own signed audit stream (the `epg-auditer` sidecar) within the fabric. *(Full EPG specification is maintained separately and folds into a future revision of this standard.)*

§21 — Distributed Verification Fabric (SentiNet)

SentiNet propagates cryptographically signed state across nodes using a **push-verify** model with local enforcement — no consensus ledger. Each node independently validates trust state without querying the origin.

- **Signed policy bundles** distributed to all nodes; verified locally (schema + policy-hash + signature + node-binding)
- Integrity verification against known-good roots at each node
- **Last-Known-Good (LKG)** fallback automatically invoked on verification failure — a bad bundle cannot brick a node
- Fail-closed enforcement when trust state cannot be confirmed
- Cross-node trust propagation without central coordination
- **Identity model:** the bundle declares **operators**, **nodes**, **services** (with ports), **zones**, and **flows** — the governed topology. Current bundle: **v2.2**, three nodes (DL360, Lenovo, ANVIL), three zones (Z1_CORE, Z2_ENFORCEMENT, Z0_ROOT), 21 declared services.
- **Signature (current / planned):** Ed25519 bundle signature today; migration to **hybrid Ed25519 × ML-DSA-87 AND-verification** is specified (§36) to bring bundle trust to the same post-quantum bar as GAT signing.

§22 — Bounded Storage Lifecycle

Three tiers; cryptographic integrity preserved regardless of medium, enabling long-term forensics without unbounded growth.

Tier	Characteristics	Verification
HOT	Active segments, immediate access, full resolution	Real-time Merkle + hash chain
WARM	Sealed, compressed, recent history	Segment root vs ledger
COLD	Archived, offline media, long retention	Offline signature + root check

§23 — Implementation Evidence

This is not theoretical. Verified in the active deployment as of this revision:

- Governance events with embedded policy/approval metadata generated in production
- Merkle-sealed segments with independently verifiable roots — validated
- Signed audit artifacts distributed across SentiNet nodes — operational (bundle v2.2)
- Distributed policy verification without central coordination — confirmed
- **Post-quantum signing (ML-DSA-87) live** — SPECTER Ghost Key emits an ML-DSA-87 signature under `require_pqc` (single-signature, fail-closed on PQC absence); the Ghost Key KEM-seed layer is hybrid (Shamir + ML-KEM-1024 via HKDF-SHA3-512)

- **Genuine hybrid AND-signature (ECDSA-P256 + ML-DSA-87) produced by the ANVIL hardware root — running in *observe* mode** alongside the legacy RSA-PSS base signature; the hybrid-required cutover is pending (§36). *The unified hybrid-AND signature is not yet the enforced GAT signature — stated plainly rather than implied as complete.*
- **Sentinel Forge governed apply — deployed** (signature-re-verifying, fail-closed, component registry)
- Metatron's Gate — ENFORCING; Sentinel enforcement — staged

Honest boundaries: enforcement is staged (§9); the platform has **not** undergone external adversarial review; it is a single-operator research deployment (§35).

§24 — Security Analysis — Threat Model

Threat	Mitigation
Log tampering	Hash-chain continuity + Merkle root binding detect any modification
Insider manipulation	Quorum approval limits unilateral action; PQC-signed artifacts
Storage corruption	Segment-level Merkle sealing localizes corruption
Ledger truncation	Chain-continuity gaps detected; SentiNet propagates last-good state
Replay attacks	Time-bounded GATs + hash-chained segment ordering (each entry binds its predecessor's hash)
Node compromise	Fail-closed enforcement + LKG fallback contain blast radius
Policy corruption	Signed policy bundles with independent verification
Single-authority capture	Multi-party quorum for sensitive decisions; signing/enforcement separated
Signing-host compromise	Keys are off-host and TPM-sealed on ANVIL; a compromised enforcement host cannot forge authority
Retroactive signature forgery (quantum)	Long-lived provenance signatures use ML-DSA-87; a future CRQC (cryptographically relevant quantum computer) that breaks a classical signature cannot forge the post-quantum half, so sealed provenance remains valid. <i>This is the dominant quantum threat to a governance/audit system — see §6 above, and §5.2 of the white paper.</i>
Harvest-now-decrypt-later (quantum)	A confidentiality threat: an adversary records ciphertext today to decrypt once a CRQC exists. Addressed by ML-KEM-1024 key encapsulation on material that must stay confidential (the Ghost Key lattice seed; the ANVIL KEM path; PQC-TLS on the roadmap). Signatures do not mitigate this — key encapsulation does.

§25 — Comparison With Blockchain

Dimension	Blockchain	LAIRS Audit Fabric
Trust model	Consensus-based	Cryptographic + governance-native, hardware-rooted
Storage growth	Unbounded	Bounded lifecycle tiers
Verification access	Requires live network	Fully offline capable
Governance coupling	External / implicit	Embedded in every event record
Latency	Consensus-dependent	Deterministic, local
Failure recovery	Fork-resolution complexity	LKG + fail-closed
Event semantics	Transaction-oriented	Decision provenance records
Infrastructure fit	Cryptocurrency / DeFi	Infrastructure telemetry + governance

§26 — Application Domains

Critical-infrastructure protection; ICS/SCADA deterministic audit; defense networks with offline-capable verification; **autonomous AI governance with provable decision provenance and human override**; sovereign cloud requiring offline verification; supply-chain cryptographic continuity.

PART III — Component Catalog

Current public architecture as of July 24, 2026. **Implemented** = operational in the current deployment; **In Development** = under active design or integration. The Human Master Administrator retains ultimate authority at all times; no component acts autonomously or irreversibly without explicit human cryptographic approval.

§27 — Components

Implemented — Operational

- **LAIRS Core** — cognitive runtime and orchestration; event routing between modules; identity/memory/state management; policy hooks, audit logging, explainability; local-first; no autonomous irreversible actions.
- **ANVIL** — hardware-root-of-trust signing appliance (§13); off-host, TPM-sealed, attestation-first; produces a hybrid ECDSA + ML-DSA-87 signature (running in **observe** mode today — §6, §23) plus ML-KEM key encapsulation.
- **Metatron's Gate** — quorum governance and authority control; intent declaration; Machine/AI/Human triad quorum; post-quantum (ML-DSA-87) GAT signing live, ANVIL-rooted **hybrid** signing in **observe** (§6, §23); append-only causal audit; gate **ENFORCING**.
- **SentiNet Fabric** — zero-trust fabric; zone segmentation with default-deny; signed policy bundles (v2.2) with local verify + LKG; node/service/zone/flow identity model; internal DNS/PKI anchoring.
- **Sentinel Agent** — host-level integrity, telemetry, local attestation; signed organic baseline; encrypted channel to Core.
- **Sentinel Forge** — human-approved governed system evolution; detects/stages candidate changes; **never self-applies**; signature-re-verifying privileged apply at controlled boundaries; component registry (governed apply to registered trees only).
- **Execution Provenance Gate (EPG)** — execution-provenance enforcement (§20); refuses execution lacking verified provenance; own signed audit stream.
- **PRAETOR** — the sovereign reasoning model (self-hosted small LM); governed disposition output routed through Metatron's Gate; reads live telemetry (no-hands) and proposes, never disposes.
- **Mnemos** — PRAETOR's persistent-memory / continuity substrate; hash-chained, genesis-anchored to the Founding Charter; identity persists across model versions under defined conditions.
- **GAE** — Governed Adversarial Evolution engine (§8); red/blue/purple exercises with human approval gates.
- **AiCraft** — local AI training and lifecycle governance; explicit datasets only; versioned artifacts; audit trail per training run.
- **Praetorian Guard** — active runtime guard with signed policy and live detectors.

In Development

- **LICTOR** — enforcement authority (§14); counterpart to ANVIL; not yet built.
- **VEDETTE** — governed OSINT collection agent (requester seat + credential-less forward executor; passive/public-only; provenance-on-ingress); prototype.
- **ANVIL production OS** — custom immutable Buildroot appliance (secure/measured boot, dm-verity, TPM-sealed, PQC); designed, not yet built.
- **Sentinel Gate** — ingress control for the fabric; policy validation and trust scoring; fail-closed on unknown traffic.

- **Sentinel AV** — sovereign local malware detection/quarantine integrated with LKG.
- **Leave No Trace (LNT)** — data minimization and forensic hygiene without compromising audit integrity.

PART IV — Deployment & Technical Reference

§28 — Node Topology

The deployment spans three systems joined by the SentiNet verification channel, with operational separation between artifact production, verification, and signing authority:

- **DL360 — Primary.** Metatron's Gate; segmented Merkle audit generation; Sentinel enforcement; artifact sealing/archival; LAIRS Core; PRAETOR/Mnemos; Sentinel Forge; GAE. Origin of governance decisions.
- **Lenovo — Secondary.** SentiNet verification endpoint; signed-artifact validation; known-good reference; independent verification environment (verification independence from the origin).
- **ANVIL — Trust Root.** Off-host signing authority on an isolated dedicated link; TPM-sealed keys; attestation. Zone Z0_ROOT.

(v1.0 labeled the primary "GL360"; the host is a DL360 — corrected here and tracked for cleanup across modules.)

§29 — Ledger Event Structure

Representative SEGMENT_SEALED event:

```
{
  "ts": 1784904961.123,
  "event_type": "SEGMENT_SEALED",
  "hash": "sha256:a3f9b12d...", // links this entry into the hash chain
  "prev_hash": "sha256:7c4e0a91...", // continuity to previous entry
  "payload_sha256": "sha256:d1f8c322...", // protects compressed artifact
  "merkle_root": "sha256:9b2e7f55...", // commits segment event contents
  "segment_basename": "segment_20260724_145601.jsonl.gz",
  "sig_basename": "segment_20260724_145601.jsonl.gz.sig"
}
```

§30 — Segment Artifact Lifecycle

Events generated by the governance engine → written sequentially to a segment JSONL → Merkle tree computed → root generated → payload gzipped + SHA-256 → **PQC signature applied by the ANVIL-rooted signing key** → ledger entry recorded (hash, prev_hash, merkle_root, payload_sha256) → stored as segment_*.jsonl.gz + .sig. Both files are required for independent verification; segments remain verifiable regardless of storage location.

§31 — Independent Verification Procedure

No access to the origin runtime is required — only the segment files, signatures, and the authority's public keys:

1. Decompress the segment artifact
2. Recompute the Merkle tree from all event records
3. Compare the computed root to merkle_root in merkle_ledger.jsonl
4. Validate the detached signature against the authority public key(s) per the signature scheme in force; under the **hybrid-AND target scheme** (§4.1), **both** halves must verify

5. Confirm hash-chain continuity: each entry's `prev_hash` == predecessor's hash
6. Validate governance metadata (policy, approval state, actor identity) per event

§32 — Governance Identity Controls

- Password hashing (PBKDF2; migration to Argon2id planned) with per-credential salts
- TOTP MFA for governance actors
- Enrollment validation with identity event logging
- Policy-controlled authentication governed by the active policy bundle
- Identity data bound to governance decisions in all audit records
- **Secret hygiene:** shared authority credentials are rotated, held in env/sealed stores (not source), fail-closed on absence.

***Legal note:** the system provides authenticated attribution but does not claim legal non-repudiation. Consult counsel for regulatory requirements in your deployment context.*

§33 — SentiNet Verification Channel — Current Behavior

Currently operational	Not yet
Signed bundle distribution + local verify (schema/hash/sig/node-binding)	Multi-peer consensus (>3 nodes), automatic mesh
Known-good state referencing + LKG fallback	Byzantine fault tolerance (BFT)
Independent verification at each node	Network egress/zone enforcement (Phase 3)
ANVIL enrolled as a governed node (Z0_ROOT)	Hybrid AND-sig bundle verification (§36)

SentiNet currently provides deterministic verification within a governed trust boundary; active network enforcement of the declared flows is Phase 3.

§34 — Storage Layout

```
/mnt/sentinet_logs/
  lairs_audit_archive/
    segment_YYYYMMDD_HHMMSS.jsonl.gz      # hot – active segments (UTC date + time)
    segment_YYYYMMDD_HHMMSS.jsonl.gz.sig  # detached signature
    merkle_ledger.jsonl                   # hash-chain ledger
    [warm/cold archived via tiered lifecycle policy]
```

Integrity is preserved through cryptographic artifacts, not storage-medium trust. All service data resides on dedicated bulk storage, never on the system disk.

§35 — Security Assumptions

- Cryptographic primitives (SHA-256/SHA-3, ML-DSA-87, ML-KEM-1024, ECDSA/Ed25519) are not compromised

- Governance signing keys are **TPM-sealed on ANVIL** and never present on enforcement hosts
- The signing appliance is reachable only over an isolated dedicated link
- Nodes operate within a governed administrative boundary with physical controls
- Simultaneous physical compromise of all nodes is unlikely under the threat model

§36 — Acknowledged Limitations & Roadmap

Current limitations (research deployment) — stated to avoid overstating maturity:

- Three-node deployment; not validated at scale; no multi-peer mesh or BFT
- Enforcement is staged; several controls run in observe mode (§9)
- Hardware-root-required GAT verification is deployed in *observe* — cutover to required is pending caller round-trip + ANVIL high-availability (HA)
- SentiNet bundle signature is single Ed25519 today; hybrid AND-sig migration pending
- No formal verification proofs for governance-protocol or GAT-lifecycle correctness
- **Not externally adversarially reviewed** — a prerequisite for any production commissioning
- Single-operator deployment

Roadmap:

- Complete the ANVIL custody cutover (hardware-root-required, drop legacy RSA) once availability hardening lands
- **Hybrid Ed25519 × ML-DSA-87 AND-verification for SentiNet bundles** — bring policy-bundle trust to the GAT bar
- Build LICTOR (enforcement authority) and the two-authority routing
- SentiNet Phase-3 enforcement (governed egress/zone) with flow permissions
- Ship the ANVIL immutable Buildroot appliance (secure/measured boot, dm-verity, TPM-sealed) with `sentinetd` + guard baked in
- Metatron's Gate tiered five-voter quorum; multi-peer BFT beyond three nodes
- Formal verification of governance-protocol correctness
- External adversarial review (crypto + AI red-team + architecture) before commissioning

Appendix A — Terminology

Term	Definition
ANVIL	<i>Attested Node, Vaulted Identity Lineage</i> — hardware-root-of-trust signing appliance; off-host, TPM-sealed, attestation-first. The Layer-0 trust anchor.
Audit Fabric	Cryptographically sealed, governance-aware logging with deterministic verification; produces decision-provenance records, not passive logs.
BFT	Byzantine Fault Tolerance — consensus correctness despite arbitrarily-faulty or malicious nodes; a scaling target beyond the current three-node deployment (§36).
CRQC	Cryptographically Relevant Quantum Computer — a quantum computer capable of breaking classical public-key cryptography (RSA / ECDSA).
EPG	Execution Provenance Gate — refuses execution whose provenance cannot be verified; extends fail-closed governance to execution.
GAE	Governed Adversarial Evolution — governed red/blue/purple improvement with cryptographic human approval gates.
GAT	<i>Governed Action Token</i> — time-bounded, context-scoped, node-bound artifact authorizing a specific action; signed post-quantum (ML-DSA-87), with hardware-rooted hybrid AND-signing as the target scheme (observe today — §4.1, §23). The universal authorization primitive. Failure $\Rightarrow$ fail-closed.
Ghost Key	Single-use SPECTER signing key assembled from Shamir shares and a hardware-KEM lattice seed, then destroyed; never persists at rest.
Hash Chain	Sequence where each entry includes its predecessor's hash; gaps indicate truncation/insertion.
Hybrid AND-signature	A signature valid only if both a classical and a post-quantum signature verify.
LICTOR	<i>Latching Interlock & Containment, Trusted Operational Response</i> — enforcement authority; counterpart to ANVIL (signing). Executes enforcement decisions; separate trust domain. (<i>In development.</i>)
LKG	Last-Known-Good state — validated fallback used when current-state verification fails; recovery without loss of auditability.

Term	Definition
Merkle Tree / Root	Hash tree over a set of events; the root commits the set; enables partial verification without full history.
Metatron's Gate	The Authority layer — policy evaluation, GAT issuance, quorum enforcement, decision recording.
ML-DSA-87 / ML-KEM-1024	FIPS-204 signature and FIPS-203 KEM; the post-quantum primitives of the SPECTER layer.
Mnemos	PRAETOR's persistent-memory / continuity substrate; hash-chained, Charter-anchored.
PQC	Post-Quantum Cryptography — algorithms designed to resist attack by a CRQC; in LAIRS, ML-DSA-87 (signatures) and ML-KEM-1024 (key encapsulation).
PRAETOR	The sovereign, self-hosted reasoning model; proposes governed dispositions, never disposes.
Runtime-Governed System	Environment where authority, behavior, and state changes are continuously validated during operation.
Segment	Bounded, sealed group of audit events with a Merkle root, signature, and ledger entry.
SentiNet	Distributed trust-propagation and verification fabric; zero-trust, zone-segmented, LKG, signed-bundle distribution.
SPECTER	<i>Stateless Protocol for Ephemeral Cryptographic Token Endorsement and Revocation</i> — the post-quantum cryptographic layer; first instantiation of the Ethereal Cryptography paradigm.
VEDETTE	Governed OSINT collection agent — requester seat + credential-less forward executor; passive, public-source only. Named for a forward mounted sentry. (<i>In development.</i>)

Appendix B — References

Ref	Title	Locator
[1]	LAIRS Standard: Runtime-Governed Systems — White Paper v2.0 (2026)	https://github.com/ASConvergenceSystems/LAIRS-Standard-Runtime_Governance-Systems
[2]	LAIRS Audit Fabric Standard (2026)	https://github.com/ASConvergenceSystems/LAIRS-Audit-Fabric-Standard
[3]	Ethereal Cryptography — SPECTER foundation; R. Aydelotte (2026)	https://github.com/ASConvergenceSystems/Ethereal_Cryptography
[4]	lairs-architecture-pub	https://github.com/ASConvergenceSystems/lairs-architecture-pub
[5]	governed-adversarial-evolution	https://github.com/ASConvergenceSystems/governed-adversarial-evolution
[6]	NIST SP 800-207 — Zero Trust Architecture (2020)	https://doi.org/10.6028/NIST.SP.800-207
[7]	RFC 9334 — Remote Attestation procedureS (RATS) Architecture (2023)	https://www.rfc-editor.org/rfc/rfc9334
[8]	in-toto — software supply-chain integrity	https://in-toto.io
[9]	The Update Framework (TUF)	https://theupdateframework.io
[10]	SPIFFE / SPIRE — workload identity	https://spiffe.io
[11]	Keylime — TPM-based remote boot & runtime attestation	https://keylime.dev

Appendix C — Related Standards and Positioning

Runtime governance is an active field. LAIRS does **not** claim to invent any single primitive below — each has strong, mature prior art, and honest positioning requires naming it. LAIRS's claim is one of **integration**: unifying these into a single continuous runtime authority loop, as a conformance standard *plus* a working reference implementation, on sovereign, offline-capable infrastructure.

Standard / System	Domain	What it provides	Relationship to LAIRS
NIST SP 800-207 [6]	Zero-trust architecture	Per-request, never-trust-always-verify access decisions	LAIRS applies zero-trust <i>at runtime</i> ; the GAT is the per-action trust decision. LAIRS adds a hardware-rooted, PQC-signed authorization artifact and an immutable provenance ledger.
RATS — RFC 9334 [7]	Remote-attestation semantics	Attester / Verifier / Relying-Party roles; evidence → attestation results	ANVIL is an Attester emitting TPM-quote evidence; LAIRS nodes are Relying Parties. LAIRS binds attestation to <i>every signature</i> , not only to node admission.
Keylime [11]	TPM runtime attestation	Continuous measured-boot + IMA (Integrity Measurement Architecture) runtime integrity	Overlaps ANVIL/EPG (measured boot, runtime integrity). LAIRS extends attestation into per-action authorization, PQC signing, and governed enforcement.
SPIFFE / SPIRE [10]	Workload identity	Attestation-based short-lived workload identities (SVIDs — SPIFFE Verifiable Identity Documents)	The GAT is action-scoped, not identity-scoped : it authorizes <i>what may happen</i> (time-bounded, intent-hashed), rather than <i>who a workload is</i> . Complementary.
in-toto [8]	Supply-chain provenance	Signed attestations that each pipeline step was performed by the authorized party	Same provenance spirit, applied to <i>runtime actions</i> rather than <i>build steps</i> ; LAIRS adds a hardware root, PQC, quorum, and continuous re-verification.

Standard / System	Domain	What it provides	Relationship to LAIRS
TUF [9]	Update distribution	Role/threshold signing; freshness and rollback protection	LAIRS's signed policy bundles + Last-Known-Good echo TUF's threshold signing and rollback protection, applied to runtime authority rather than package delivery.

Positioning. No individual mechanism here is novel. What is uncommon is the *assembly*: hardware-rooted attestation (RATS/Keylime-class) **and** per-action, PQC-signed authorization tokens (a GAT — the provenance spirit of in-toto, but for actions) **and** zero-trust re-verification at every hop (800-207) **and** threshold/rollback-protected policy distribution (TUF-class) **and** a strict separation of signing authority (ANVIL, deployed) from enforcement authority (LICTOR, in development; enforcement runs on separate components today) — expressed as a measurable conformance standard with a running reference implementation on single-operator, offline-capable infrastructure. That integration is the contribution; the primitives are prior art, and are cited as such.

Open, non-proprietary, non-confidential convergence of the LAIRS Runtime Governance Architecture and Audit Fabric Standard into a single unified public technical reference. Approved for public release under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International. Copyright © 2026 Russell Aydelotte, Aegis Secure Convergence Systems. <https://creativecommons.org/licenses/by-nc-sa/4.0>